

Hands-On-Learning: Refactor a Monolithic API into Domain-Driven Microservices

Learning Objectives

By the end of this activity, you will be able to:

- Analyze API request patterns to identify domain-driven service boundaries.
- Configure essential API Gateway routing and rate-limiting rules.
- Implement a minimal database-per-service migration plan.
- Evaluate scalability improvements using a lightweight local simulation.



Description

In this hands-on practice, you will refactor a simplified monolithic API into a set of domain-driven microservices by analyzing request patterns, configuring an API Gateway, and validating scalability improvements through a lightweight local simulation.

Structure of the Activity

Title: Refactor a Monolithic API into Domain-Driven Microservices

Dataset Information

→ **JSON dataset** (techmart_request_logs_sample.json)

→ **Dimensions:** Approximately 50–200 request records (small synthetic subset for rapid analysis).

→ **Key Features:**

→ endpoint, method, response_time_ms, status_code

→ database_queries, db_time_ms, service

→ external_calls (optional list for dependency mapping)

Tools used:

Software / Platform:

→ Python 3.10 (local environment or cloud notebook)

Required Libraries:

→ flask (version 3.x) — for minimal mock services

→ pandas (version 2.x) — for loading and inspecting logs

→ requests (version 2.x) — for simple routing simulation

→ Standard library modules: json, time, random, typing

Setup Requirements

Technical Prerequisites:

- Basic Python coding skills
- Ability to run a Python script or notebook locally

System Requirements:

- Any OS with Python 3.10 installed
- At least 2GB RAM
- Internet not required

Required Files in Working Directory:

techmart_request_logs_sample.json (provided or generated in Getting Started)

Scenario

TechMart is experiencing performance bottlenecks in its monolithic API, causing slow responses for high-traffic endpoints such as user login and product search. Engineering leadership wants to validate whether splitting the monolith into domain-focused microservices can improve scalability. You will act as the platform engineer responsible for analyzing API request patterns, designing service boundaries, and configuring a basic API Gateway routing strategy to demonstrate how microservices improve performance.

Objective

Create a minimal, functional microservices simulation that identifies service boundaries, routes calls through a lightweight gateway, and demonstrates improved performance using a database-per-service pattern.



Getting Started:

1

Create a new folder for this practice activity and open a Python notebook or script inside it.

2

Place the sample dataset (techmart_request_logs_sample.json) into the same directory—or generate it using the provided snippet in Activity 1.

3

Verify your environment by running a test cell that imports flask, pandas, and requests to confirm your setup is ready.

Activity 1: Analyze Request Patterns and Identify Service Boundaries

Objective

Analyze the dataset to identify natural domain-driven service boundaries within the TechMart monolith.



Input/Data

techmart_request_logs_sample.json

No previous activity output required

Essential imports (only if needed):

```
import pandas as pd
```

```
import json
```

Steps to Complete

Step 1: Load the dataset using a standard JSON or CSV reading function.

Step 2: Print the top endpoints by request frequency and average database time to reveal high-traffic and high-cost operations.

Step 3: Group API endpoints into logical domains (for example, user, product, recommendations).

Step 4: Select two or three domains that show clear separation based on request patterns and business function.

Step 5: Write a short justification (two to three sentences) explaining why these boundaries are suitable for microservices.

Expected Output

A simple printed summary showing the most frequently used endpoints.

A grouped list of at least two service domains (for example, "user-service", "product-service").

A short explanation describing why these boundaries make sense.

Success indicator: Clear, logical domain separation supported by dataset patterns.

Tips

If the dataset looks too large, filter it to the first 100 rows for faster inspection.

Watch for endpoints with similar URL prefixes—they often represent the same domain.

Activity 2: Configure Lightweight API Gateway Routing

Objective

Create minimal routing rules that forward requests to your domain-driven microservices using a simplified gateway pattern.



Input/Data

Domain list created in Activity 1

No additional files required

Essential imports (only if needed):
from flask import Flask, jsonify, request

Steps to Complete

Step 1: Create a small Flask app to act as a mock API Gateway.

Step 2: Define routing rules that forward incoming paths (for example, `/api/users/*`) to the correct microservice handler.

Step 3: Implement two lightweight microservice handlers inside the same file (for example, `user_service()` and `product_service()`) to simulate responses.

Step 4: Add simple rate-limiting logic (for example, count requests in a dictionary and limit after a threshold).

Step 5: Send two or three test requests using Python's `requests` module or the browser to confirm routing works correctly.

Expected Output

A minimal API Gateway app running locally.

Correct routing of requests to the corresponding mock microservice handler.

A basic printout or log confirming rate-limiting behavior.

Success indicator: Test calls return the correct simulated responses.

Tips

Keep routing simple—regular if statements or string prefix checks are enough.

If Flask shows port errors, stop previous runs or change the port number.

Activity 3: Simulate Database-per-Service and Validate Performance Improvements

Objective

Simulate a basic database-per-service migration and compare monolith vs. microservices performance using a simple local test.



Input/Data

Service boundaries from Activity 1

Gateway logic from Activity 2

Essential imports (only if needed):

import time

import random

Steps to Complete

Step 1: Create a small function that simulates monolithic request handling by adding delays for shared database contention.

Step 2: Create a second function that simulates independent service databases with reduced delay.

Step 3: Run a loop of 50–100 simulated requests for both approaches, measuring average response time.

Step 4: Calculate and print simple metrics, such as average latency or percentage improvement.

Step 5: Write one or two sentences describing how database-per-service improves scalability.

Expected Output

Printed average latency values for “monolith” and “microservices” simulations.

A short computed improvement percentage (no exact numbers required).

A clear written statement summarizing observed performance benefits.

Success indicator: Microservices simulation shows noticeable latency improvement.

Tips

Randomized delays make simulations feel realistic—use small ranges like 10–50 ms.

If response times look identical, increase the monolith delay to better visualize contention.

Compile Your Findings

1 Requirement: Service Boundary Identification

Description: Demonstrate your ability to analyze request patterns and define clear microservice domains.

Include:

Code that loads the sample dataset and prints to request frequencies and average database times.

A grouped list of service domains (for example, user-service and product-service).

A short-written explanation (two to three sentences) demonstrates understanding of why these boundaries are appropriate for microservices.

2 Requirement: API Gateway Routing Implementation

Description: Show that you can implement functional routing logic that directs requests to the correct microservice handlers.

Include:

- A minimal Flask-based gateway with routing rules for at least two domains.
- A simple rate-limiting mechanism (counter or threshold).
- Test call results or printed logs showing that requests were routed to the appropriate mock microservice.

Evidence that your routing logic executes without dependency on external files.

3 Requirement: Performance Simulation and Analysis

Description: Demonstrate understanding of scalability differences between monolithic and microservices architectures through a simple simulation.

Include:

- Code that simulates monolith delays versus microservice delays.
- Calculated average latency for both approaches and a percentage improvement.



A brief interpretation (two to three sentences) explaining why database-per-service reduces contention.

A final statement addressing the business implication of the results (for example, “Microservices would improve customer-facing performance under high traffic”).

Summary

This activity teaches the core skill of **refactoring a monolithic API into domain-driven microservices** through practical implementation. You have learned how to analyze API request patterns, design clear service boundaries, configure lightweight gateway routing, and validate scalability improvements using a simple performance simulation. These skills directly apply to modern backend engineering environments where organizations migrate from tightly coupled monoliths to independently deployable microservices for reliability and scale.

Resources

Dataset: techmart_request_logs_sample.json

Documentation:

- Flask Docs: <https://flask.palletsprojects.com>
- Python Requests Docs: <https://requests.readthedocs.io>
- Fundamentals of Microservices Architecture (general reading)

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.